Machine Learning 2 - HW 2

Submission Instructions

This notebook contains all the questions. You should follow the instructions below. Solutions for both theoretical and practical parts should be written in this notebook.

You should submit three files:

- IPYNB notebook:
  - All the wet and dry parts, including code, graphs, discussion, etc.
- Model weights:
  - Export the model's weights after training using a `.pt` or `.pkl` file (question 2).
- PDF file:
  - Export the notebook to PDF. Make sure that all the cells are visible.

All files should be in the following format: "HW2_ID1_ID2.file"

Good Luck!

Question 1 - Generalization and Overfit (20 pt)

In this exercise, we will demonstrate overfitting to random labels. The settings are the following:

Use the MNIST dataset.

For model training - work on the first 128 samples from the train split.

Fix the following parameters:

- Shuffle to False.
- Batch size to 128.
- Use seed = 42 for reproducibility.

Generate random labels from Bernoulli distribution with a probability of $\frac{1}{2}$ (I.e., each sample is assigned a random label which is zero or one).

Show that by using a simple Fully Connected netwrok and cross-entropy loss, you are able to achieve a loss value of ≈ 0 (the lower the better).

Plot the accuracy and loss convergence for this data and the test split of our dataset as a function of epochs.

Answer the following questions:

1. Describe the accuracy value of the test data as a function of epochs and explain the phenomenon.

2. Discuss the connection between model size (complexity) and the dataset size. How does it relate to generalization and overfitting?

3. Discuss general methods to avoid overfitting. Specially, discuss methods in three different levels: data, model, and training.

4. Find one academic paper that presents or analyzes a method for reducing overfitting in deep learning.

   The method may be: general or domain-specific, architecture-specific (e.g., CNNs, Transformers), or applied to a particular task (e.g., vision, NLP, biomedical data).

   Briefly summarize:

   - The overfitting problem addressed in the paper (if there is one)
   - The proposed method for reducing overfitting
   - The main findings or conclusions

     Include a link for the paper.

     For example, you might delve into the connection between model size (complexity) and the dataset size stated in https://arxiv.org/pdf/2001.08361 </p>

In [6]:

```
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# Settings
# ----------------------------
seed = 42
torch.manual_seed(seed)
np.random.seed(seed)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

batch_size = 128
shuffle = False
N = 128          # first 128 train samples
epochs = 500     # enough to reach loss ~ 0
p = 0.5          # Bernoulli probability (1/2)

# ----------------------------
# MNIST dataset
# ----------------------------
transform = transforms.Compose([
    transforms.ToTensor(),  # converts to [0,1]
])

# Train split
train_dataset = datasets.MNIST(root="./data", train=True, download=True, transform=
```

```
train_x = train_dataset.data[:N].float() / 255.0      # [N,28,28]
train_x = train_x.unsqueeze(1)                        # [N,1,28,28]
train_y = torch.from_numpy(np.random.binomial(1, p, size=N)).long()

train_ds = torch.utils.data.TensorDataset(train_x, train_y)
train_loader = torch.utils.data.DataLoader(train_ds, batch_size=batch_size, shuffle

# Test split
test_dataset = datasets.MNIST(root="./data", train=False, download=True, transform=

M = len(test_dataset.data)  # full test set (10000)
test_x = test_dataset.data[:M].float() / 255.0
test_x = test_x.unsqueeze(1)
test_y = torch.from_numpy(np.random.binomial(1, p, size=M)).long()

test_ds = torch.utils.data.TensorDataset(test_x, test_y)
test_loader = torch.utils.data.DataLoader(test_ds, batch_size=batch_size, shuffle=F

# ----------------------------
# Simple Fully Connected Network
# ----------------------------
class FCNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Flatten(),              # [B,1,28,28] -> [B,784]
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Linear(128, 2)          # 2 classes: 0/1
        )

    def forward(self, x):
        return self.net(x)

model = FCNet().to(device)

# ----------------------------
# Loss + Optimizer
# ----------------------------
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

# ----------------------------
# Lists for convergence plots
# ----------------------------
train_losses = []
test_losses = []
train_accuracies = []
test_accuracies = []

# ----------------------------
# Train + Test loop
# ----------------------------
for epoch in range(epochs):
    # ----- Train -----
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for xb, yb in train_loader:
```

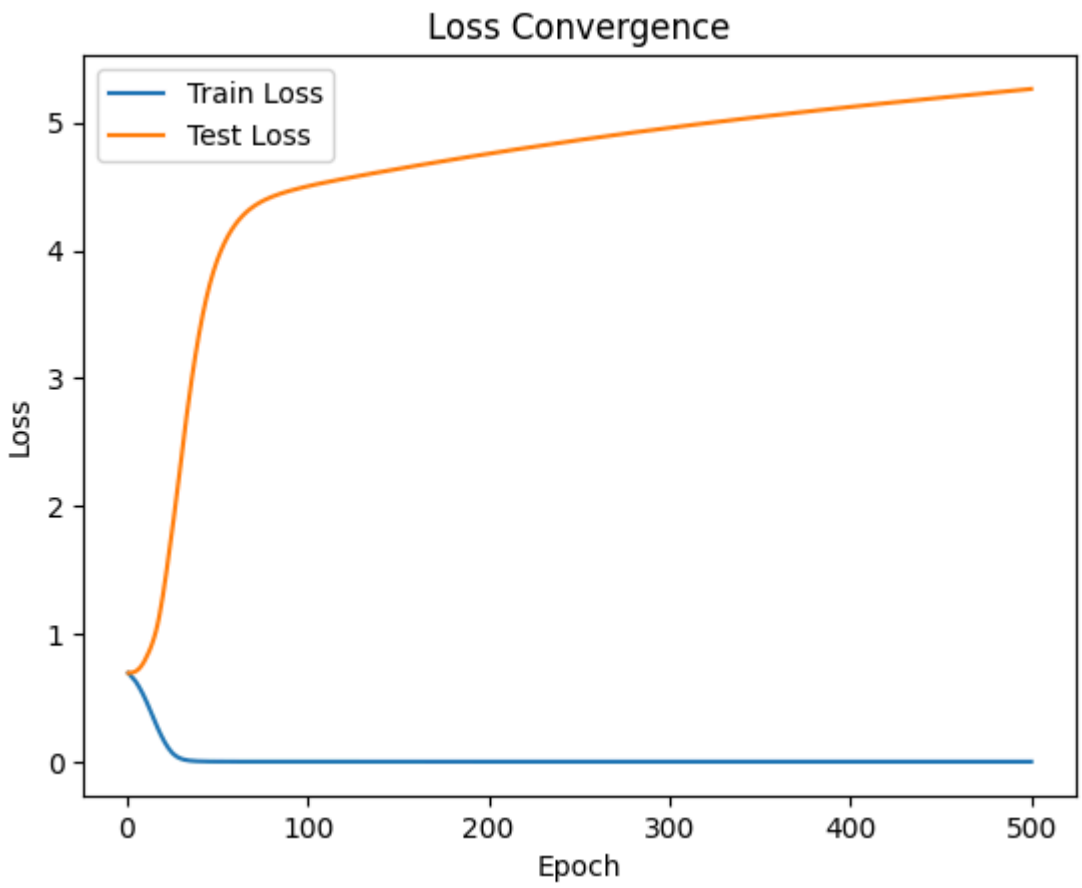
```python
        xb, yb = xb.to(device), yb.to(device)

        optimizer.zero_grad()
        logits = model(xb)
        loss = criterion(logits, yb)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        preds = torch.argmax(logits, dim=1)
        total += yb.size(0)
        correct += (preds == yb).sum().item()

    train_loss = running_loss / len(train_loader)
    train_acc = correct / total

    train_losses.append(train_loss)
    train_accuracies.append(train_acc)

    # ----- Test -----
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for xb, yb in test_loader:
            xb, yb = xb.to(device), yb.to(device)
            logits = model(xb)
            loss = criterion(logits, yb)

            running_loss += loss.item()

            preds = torch.argmax(logits, dim=1)
            total += yb.size(0)
            correct += (preds == yb).sum().item()

    test_loss = running_loss / len(test_loader)
    test_acc = correct / total

    test_losses.append(test_loss)
    test_accuracies.append(test_acc)

    if (epoch + 1) % 20 == 0 or epoch == 0:
        print(
            f"Epoch {epoch+1:3d}/{epochs} | "
            f"Train Loss: {train_loss:.6f} | Train Acc: {train_acc:.4f} | "
            f"Test Loss: {test_loss:.6f} | Test Acc: {test_acc:.4f}"
        )

print("Final Train Loss:", train_losses[-1])
```

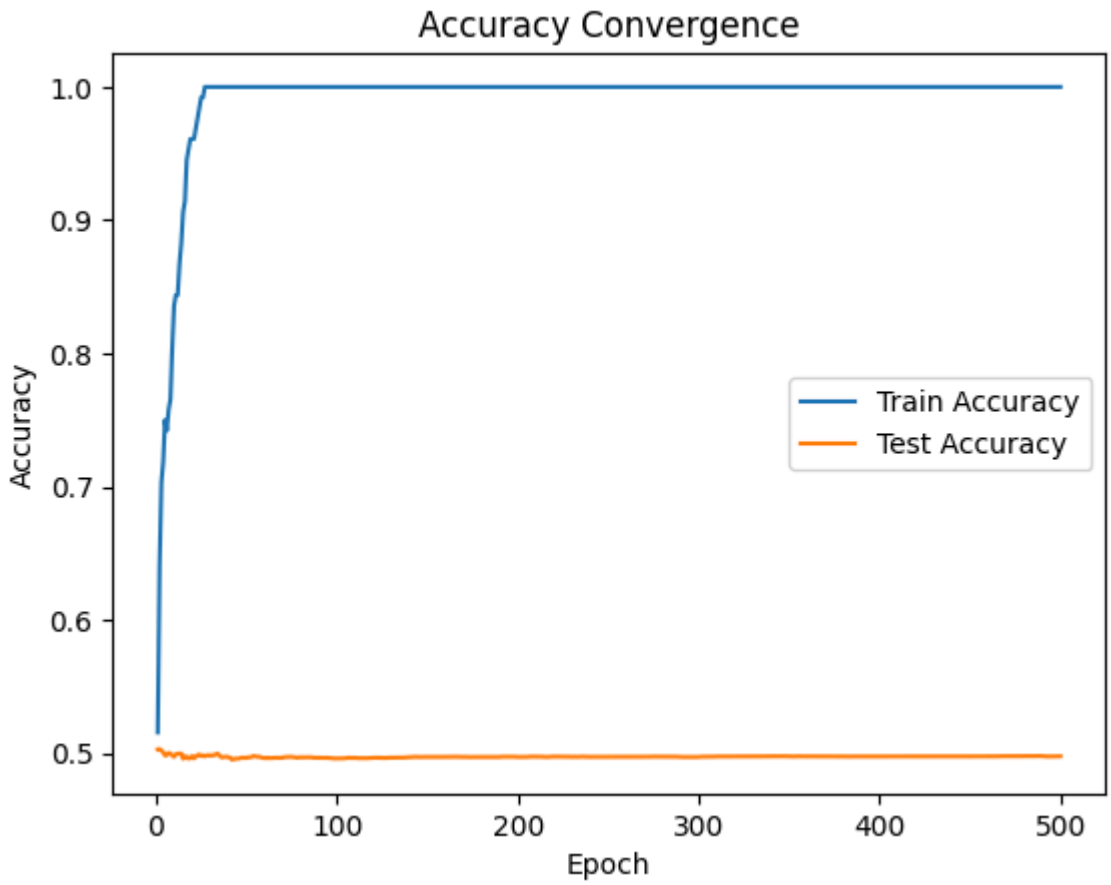
```
Epoch   1/500 | Train Loss: 0.692975 | Train Acc: 0.5156 | Test Loss: 0.693918 | T
est Acc: 0.5028
Epoch  20/500 | Train Loss: 0.178792 | Train Acc: 0.9609 | Test Loss: 1.298983 | T
est Acc: 0.4979
Epoch  40/500 | Train Loss: 0.003292 | Train Acc: 1.0000 | Test Loss: 3.346054 | T
est Acc: 0.4969
Epoch  60/500 | Train Loss: 0.000405 | Train Acc: 1.0000 | Test Loss: 4.201448 | T
est Acc: 0.4965
Epoch  80/500 | Train Loss: 0.000209 | Train Acc: 1.0000 | Test Loss: 4.417420 | T
est Acc: 0.4969
Epoch 100/500 | Train Loss: 0.000158 | Train Acc: 1.0000 | Test Loss: 4.501117 | T
est Acc: 0.4960
Epoch 120/500 | Train Loss: 0.000130 | Train Acc: 1.0000 | Test Loss: 4.559661 | T
est Acc: 0.4966
Epoch 140/500 | Train Loss: 0.000110 | Train Acc: 1.0000 | Test Loss: 4.612178 | T
est Acc: 0.4970
Epoch 160/500 | Train Loss: 0.000094 | Train Acc: 1.0000 | Test Loss: 4.661908 | T
est Acc: 0.4972
Epoch 180/500 | Train Loss: 0.000081 | Train Acc: 1.0000 | Test Loss: 4.709753 | T
est Acc: 0.4971
Epoch 200/500 | Train Loss: 0.000071 | Train Acc: 1.0000 | Test Loss: 4.755647 | T
est Acc: 0.4971
Epoch 220/500 | Train Loss: 0.000062 | Train Acc: 1.0000 | Test Loss: 4.799478 | T
est Acc: 0.4974
Epoch 240/500 | Train Loss: 0.000055 | Train Acc: 1.0000 | Test Loss: 4.842061 | T
est Acc: 0.4972
Epoch 260/500 | Train Loss: 0.000049 | Train Acc: 1.0000 | Test Loss: 4.882793 | T
est Acc: 0.4971
Epoch 280/500 | Train Loss: 0.000044 | Train Acc: 1.0000 | Test Loss: 4.921558 | T
est Acc: 0.4973
Epoch 300/500 | Train Loss: 0.000039 | Train Acc: 1.0000 | Test Loss: 4.958375 | T
est Acc: 0.4971
Epoch 320/500 | Train Loss: 0.000036 | Train Acc: 1.0000 | Test Loss: 4.993844 | T
est Acc: 0.4976
Epoch 340/500 | Train Loss: 0.000033 | Train Acc: 1.0000 | Test Loss: 5.027719 | T
est Acc: 0.4976
Epoch 360/500 | Train Loss: 0.000030 | Train Acc: 1.0000 | Test Loss: 5.060187 | T
est Acc: 0.4976
Epoch 380/500 | Train Loss: 0.000027 | Train Acc: 1.0000 | Test Loss: 5.091582 | T
est Acc: 0.4976
Epoch 400/500 | Train Loss: 0.000025 | Train Acc: 1.0000 | Test Loss: 5.121909 | T
est Acc: 0.4975
Epoch 420/500 | Train Loss: 0.000023 | Train Acc: 1.0000 | Test Loss: 5.151383 | T
est Acc: 0.4976
Epoch 440/500 | Train Loss: 0.000021 | Train Acc: 1.0000 | Test Loss: 5.180153 | T
est Acc: 0.4975
Epoch 460/500 | Train Loss: 0.000020 | Train Acc: 1.0000 | Test Loss: 5.207571 | T
est Acc: 0.4976
Epoch 480/500 | Train Loss: 0.000018 | Train Acc: 1.0000 | Test Loss: 5.234997 | T
est Acc: 0.4976
Epoch 500/500 | Train Loss: 0.000017 | Train Acc: 1.0000 | Test Loss: 5.262341 | T
est Acc: 0.4977
Final Train Loss: 1.7005444533424452e-05
```

In [7]:

```
epochs_range = range(1, epochs + 1)

plt.figure()
plt.plot(epochs_range, train_losses, label="Train Loss")
plt.plot(epochs_range, test_losses, label="Test Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss Convergence")
plt.legend()
plt.show()
```

```
plt.figure()
plt.plot(epochs_range, train_accuracies, label="Train Accuracy")
plt.plot(epochs_range, test_accuracies, label="Test Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Accuracy Convergence")
plt.legend()
plt.show()
```





1) Test accuracy as a function of epochs and explanation

The test accuracy remains approximately 0.5 (50%) throughout training, with only minor fluctuations and no consistent improvement as the number of epochs increases. This value corresponds to chance-level performance in a binary classification task.

This behavior occurs because the training labels are randomly assigned and therefore contain no meaningful relationship between the input images and the labels. As a result, the model cannot learn a generalizable pattern. While the network successfully memorizes the small training set (leading to near-zero training loss and 100% training accuracy), it is unable to generalize to unseen data. Consequently, its predictions on the test set are equivalent to random guessing.

The simultaneous increase in test loss indicates that the model becomes increasingly confident in its incorrect predictions, which is a hallmark of severe overfitting when using cross-entropy loss.

2) Connection between model size, dataset size, generalization, and overfitting

There is a strong relationship between model complexity (number of parameters) and dataset size. When a model is large relative to the amount of available training data, it has sufficient capacity to memorize the training samples rather than learn generalizable patterns. This typically results in overfitting, characterized by excellent training performance and poor test performance.

Conversely, if the model is too small, it may lack the capacity to fit the training data, leading to underfitting. Effective generalization usually occurs when the model complexity is

appropriately matched to the dataset size and diversity. In this experiment, the dataset is extremely small (128 samples) and the labels are random, making generalization fundamentally impossible regardless of the training duration.

3) General methods to avoid overfitting

Data-level methods:

Overfitting occurs when the model sees too few examples and can memorize them. Increasing the size of the training set makes memorization harder and forces the model to learn patterns that repeat across many samples. Data augmentation artificially increases the dataset by applying transformations (e.g., rotations or noise), exposing the model to slightly different versions of the same data and reducing sensitivity to exact inputs. Correcting noisy or incorrect labels is essential because label noise directly encourages memorization of errors. Balancing class distributions prevents the model from overfitting to frequent classes while ignoring rare ones.

Model-level methods:

A model with too many parameters can easily memorize small datasets. Reducing model size limits this memorization capacity. L2 regularization (weight decay) discourages large weights, favoring simpler decision boundaries. Dropout randomly disables neurons during training, preventing the network from relying on specific paths and reducing co-adaptation. Using architectures designed for the data domain (e.g., CNNs for images) introduces structural constraints that reduce unnecessary flexibility.

Training-level methods:

Early stopping prevents the model from continuing to train after it has started memorizing the training data. Learning rate scheduling reduces step sizes during training, helping convergence to smoother solutions. Keeping strict separation between training, validation, and test sets ensures that evaluation reflects true generalization. Batch normalization stabilizes training dynamics and introduces a regularizing effect that can reduce overfitting.

4) Academic paper on reducing overfitting

Paper: Dropout: A Simple Way to Prevent Neural Networks from Overfitting Authors: Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov Year: 2014 Link: https://jmlr.org/papers/v15/srivastava14a.html

Overfitting Problem Addressed:

Deep neural networks with a large number of parameters often suffer from overfitting, where the model achieves very low training error but performs poorly on unseen data. This happens because neurons can become highly dependent on specific patterns in the training set (co-adaptation).

Proposed Method:

The paper introduces Dropout, a regularization technique in which a random subset of neurons is temporarily removed during each training iteration. This forces the network to

learn more robust and generalizable feature representations instead of memorizing the training data.

Main Findings:

The authors show that Dropout significantly reduces overfitting and improves generalization across a wide range of tasks and architectures. The method is simple to implement, computationally efficient, and consistently leads to better performance on test data compared to standard training without Dropout.

Question 2 - Sentiment Analysis - Classification (70 pt)

The goal of this exercise is to get familiar with recurrent neural networks.

Text emotion recognition, AKA sentiment analysis, is a rapidly growing field due to its practical applications. For instance, companies may be interested in classifying customer reviews as positive or negative to understand user satisfaction.

In this exercise, you will detect the emotion of a sentence. You will implement two models - RNN and LSTM. You need to reach at least 51% and 70% accuracy on the test set with RNN-based and LSTM-based models, respectively.

You should:

Use different optimization and regularization methods.
Try different combinations of hyperparamters.
</ul> </p>

Data

You will work with IMDB dataset, containing movie reviews and their labels according to the emotion conveyed in the review [positive, negative]. The dataset is divided into train and test sets (folders), each contains a folder for positive and negative reviews.

The dataset vocabulary is given (`imdb.vocab`). Feel free to use it or any other custom vocabulary.

Tips

- Instead of using One-hot embeddings for the data, use `nn.Embedding` .
- Tokenization ("model language for text") - use word-based tokenization (each word is a token).

- You also might add "Start Of Sentence" (SOS) and "End Of Sentence" (EOS), "padding" (PAD) and "unknown" (UNK) tokens (self-learning). Note, to use them you have to insert them manually to your vocabulary and to the dataset's text.

  You are given `VocabEmbedding` for word-embedding management. The class loads the dataset's vocabulary and assigns an embedding for each word in it (+ special tokens). Feel free to use it as part of your model. Note, the embeddings are trainable and the class is a torch.nn class.

- Use GPU.

- It's recommended to use Deep RNN\LSTM (number of recurrent layers stacked on top of each other). For example, using `num_layers` in `nn.RNN`.

a) EDA (10 pt)

Explore and analyze your data. Explain your data cleaning and processing pipeline.

In [12]:
```
import os
import re
from collections import Counter
import numpy as np

# ========================
# Helper: text cleaning
# ========================
def clean_text(text):
    """
    Basic text cleaning:
    - Lowercase
    - Remove HTML tags
    - Remove punctuation & digits
    - Collapse whitespace
    """
    text = text.lower()
    text = re.sub(r"<.*?>", " ", text)        # remove HTML
    text = re.sub(r"[^a-z\s]", " ", text)     # keep letters only
    text = re.sub(r"\s+", " ", text).strip()  # normalize spaces
    return text


# ========================
# EDA starts here
# ========================
print("IMDB Sentiment Analysis: EDA \n")

# ---- Data sizes ----
print(" DATA SHAPES")
print(f"Train samples: {len(train_texts)}")
print(f"Test  samples: {len(test_texts)}\n")

# ---- Label distribution ----
print("LABEL DISTRIBUTION (0 = negative, 1 = positive)")
print("Train:", Counter(train_labels))
print("Test :", Counter(test_labels), "\n")

# ---- Review length statistics ----
train_lengths = [len(t.split()) for t in train_texts]
test_lengths  = [len(t.split()) for t in test_texts]
```

```python
print("REVIEW LENGTH STATISTICS (words)")
print(f"Train → mean: {np.mean(train_lengths):.1f}, "
      f"median: {np.median(train_lengths)}, "
      f"max: {np.max(train_lengths)}")
print(f"Test  → mean: {np.mean(test_lengths):.1f}, "
      f"median: {np.median(test_lengths)}, "
      f"max: {np.max(test_lengths)}\n")

# ---- Cleaning preview ----
print("CLEANING PREVIEW (raw → cleaned)")
for i in [0, 1]:
    raw = train_texts[i][:300].replace("\n", " ")
    cleaned = clean_text(train_texts[i])[:300]
    print(f"\n--- Example {i+1} ---")
    print("Raw:     ", raw)
    print("Cleaned:", cleaned)

# ---- Vocabulary usage (before / after cleaning) ----
print("\n VOCABULARY SNAPSHOT")
raw_tokens = Counter(" ".join(train_texts[:2000]).split())
clean_tokens = Counter(" ".join(clean_text(t) for t in train_texts[:2000]).split())

print(f"Unique tokens (raw)    : {len(raw_tokens)}")
print(f"Unique tokens (cleaned): {len(clean_tokens)}")

print("\n EDA finished.")
```

```
IMDB Sentiment Analysis: EDA

 DATA SHAPES
Train samples: 25000
Test  samples: 25000

LABEL DISTRIBUTION (0 = negative, 1 = positive)
Train: Counter({1: 12500, 0: 12500})
Test : Counter({1: 12500, 0: 12500})

REVIEW LENGTH STATISTICS (words)
Train → mean: 233.8, median: 174.0, max: 2470
Test  → mean: 228.5, median: 172.0, max: 2278

CLEANING PREVIEW (raw → cleaned)

--- Example 1 ---
Raw:      This is a very fine and poetic story. Beautiful scenery. Magnificent mus
ic score. I've been twice in Japan last year and the movie gave me this typical Ja
panese feeling. The movement of the camera is superb, as well as the actors. It go
es deep into your feelings without becoming melodramatic. Japane
Cleaned: this is a very fine and poetic story beautiful scenery magnificent music
score i ve been twice in japan last year and the movie gave me this typical japane
se feeling the movement of the camera is superb as well as the actors it goes deep
into your feelings without becoming melodramatic japanese peop

--- Example 2 ---
Raw:      I thought that this was an absolutely charming movie centering around th
e lives of Mary-Kate's and Ashley's characters Sam and Emma Stanton! They are both
trying to make both themselves and their parents happy but; unfortunately, it's ju
st not that easy for them to actually do! I thought that this w
Cleaned: i thought that this was an absolutely charming movie centering around the
lives of mary kate s and ashley s characters sam and emma stanton they are both tr
ying to make both themselves and their parents happy but unfortunately it s just n
ot that easy for them to actually do i thought that this was a

 VOCABULARY SNAPSHOT
Unique tokens (raw)    : 55650
Unique tokens (cleaned): 24983

 EDA finished.
```

Explore & Analyze The IMDB dataset contains 50,000 movie reviews, evenly split into training and test sets, with balanced sentiment labels (positive and negative). The reviews vary significantly in length, ranging from very short texts to long reviews with several thousand words. This motivates sequence modeling with padding and truncation.

Data Cleaning Each review is cleaned using a lightweight text preprocessing pipeline:

- All text is converted to lowercase to reduce vocabulary size.
- HTML tags (e.g. `<br />`) are removed.
- Punctuation, digits, and non-alphabetic characters are discarded.
- Extra whitespace is collapsed.

This cleaning step improves robustness while preserving semantic content.

Tokenization & Vocabulary The cleaned text is tokenized at the word level. The dataset vocabulary (`imdb.vocab`) may be used, along with special tokens such as:

- `<PAD>` for padding

- `<UNK>` for out-of-vocabulary words This allows handling variable-length sequences and unseen words.

Preparation for Modeling After preprocessing, reviews are converted into sequences of word indices, padded or truncated to a fixed length. The processed data is then ready for embedding layers and sequence models such as RNN and LSTM.

---

b) Main (50 pt)

Define 2 models, as requested. Train and eval them.

- Plot the gated model's accuracy and loss (both on train and test sets) as a function of epochs.
- Plot a confusion matrix for the model outputs on the train and test sets.
- Did you observe differences in models training process?

Note, you are allowed to use `nn.RNN` and `nn.LSTM` in your model.

In [13]:
```
import os, re, math, random
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torch.nn.utils.rnn import pad_sequence, pack_padded_sequence
from collections import defaultdict
import matplotlib.pyplot as plt

# ====== Reproducibility ======
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)
```

```
Device: cuda
```

In [14]:
```
import torch.nn as nn
import torch

class VocabEmbedding(nn.Module):
    """
    Handles:
      - loading vocabulary from imdb.vocab
      - mapping word <-> index
      - embedding lookup
      - providing a method to get embedding for a single word
    """
    def __init__(self, vocab_path, embed_dim=128):
        super().__init__()

        # Load raw vocab file
        with open(vocab_path, "r", encoding="utf-8") as f:
            words = [line.strip() for line in f if line.strip()]
```

```python
        # Add special tokens
        self.special_tokens = ["<pad>", "<unk>", "<sos>", "<eos>"]
        vocab = self.special_tokens + words
        word_to_token_index = {w: i for i, w in enumerate(vocab)}

        self.vocab = vocab
        self.word_to_token_index = word_to_token_index
        self.vocab_size = len(vocab)

        # Embedding layer
        self.embedding = nn.Embedding(
            num_embeddings=self.vocab_size,
            embedding_dim=embed_dim,
            padding_idx=self.word_to_token_index["<pad>"],
        )

    def forward(self, x_ids):
        """
        x_ids: LongTensor of shape [B, T] with token indices
        returns: [B, T, E]
        """
        return self.embedding(x_ids)

    def word_to_id(self, word):
        return self.word_to_token_index.get(word, self.word_to_token_index["<unk>"]

    def id_to_word(self, idx):
        return self.vocab[idx]

    def get_word_embedding(self, word):
        """
        Returns the embedding vector for a single word (1D tensor of size [E]).
        """
        self.eval()
        idx = torch.tensor([self.word_to_id(word)], dtype=torch.long, device=self.e
        with torch.no_grad():
            emb = self.embedding(idx)  # [1, E]
        return emb.squeeze(0)         # [E]
```

In [15]:

```python
TOKEN_RE = re.compile(r"[A-Za-z']+")

def tokenize(text: str):
    # IMDB מספיק <- באנגלית word-tokenization פשוט
    return TOKEN_RE.findall(text.lower())

class IMDBDataset(Dataset):
    def __init__(self, texts, labels, vocab_embed: VocabEmbedding, max_len=200):
        self.texts = texts
        self.labels = labels
        self.v = vocab_embed
        self.max_len = max_len
        self.pad_id = self.v.word_to_id("<pad>")
        self.sos_id = self.v.word_to_id("<sos>")
        self.eos_id = self.v.word_to_id("<eos>")

    def encode(self, text):
        toks = tokenize(text)
        ids = [self.sos_id] + [self.v.word_to_id(w) for w in toks] + [self.eos_id]
        if len(ids) > self.max_len:
            ids = ids[:self.max_len]
        return torch.tensor(ids, dtype=torch.long)

    def __len__(self):
        return len(self.texts)
```

```
    def __getitem__(self, i):
        x = self.encode(self.texts[i])
        y = int(self.labels[i])  # 0/1
        return x, y

def collate_batch(batch):
    xs, ys = zip(*batch)
    lengths = torch.tensor([len(x) for x in xs], dtype=torch.long)
    xs_padded = pad_sequence(xs, batch_first=True, padding_value=pad_id)  # pad_id
    ys = torch.tensor(ys, dtype=torch.long)
    return xs_padded, lengths, ys
```

In [16]:

```
VOCAB_PATH = os.path.join("aclImdb", "imdb.vocab")
EMBED_DIM = 128
MAX_LEN = 200
BATCH_SIZE = 64

vocab_embed = VocabEmbedding(VOCAB_PATH, embed_dim=EMBED_DIM).to(device)
pad_id = vocab_embed.word_to_id("<pad>")

train_ds = IMDBDataset(train_texts, train_labels, vocab_embed, max_len=MAX_LEN)
test_ds  = IMDBDataset(test_texts,  test_labels,  vocab_embed, max_len=MAX_LEN)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True,
                          collate_fn=collate_batch, num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_ds, batch_size=BATCH_SIZE, shuffle=False,
                          collate_fn=collate_batch, num_workers=2, pin_memory=True)

print("Train batches:", len(train_loader), "Test batches:", len(test_loader))
```

```
Train batches: 391 Test batches: 391
```

In [17]:

```
class RNNClassifier(nn.Module):
    def __init__(self, vocab_embed: VocabEmbedding, hidden_size=128, num_layers=1,
        super().__init__()
        self.embedding = vocab_embed.embedding  # share embedding module
        self.rnn = nn.RNN(
            input_size=self.embedding.embedding_dim,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0,
            nonlinearity="tanh"
        )
        self.drop = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_size, 2)

    def forward(self, x_ids, lengths):
        emb = self.embedding(x_ids)  # [B,T,E]
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True, enforce
        _, h_n = self.rnn(packed)  # h_n: [L,B,H]
        last = h_n[-1]             # [B,H]
        out = self.fc(self.drop(last))
        return out

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_embed: VocabEmbedding, hidden_size=128, num_layers=2,
        super().__init__()
        self.embedding = vocab_embed.embedding
        self.bidirectional = bidirectional
        self.lstm = nn.LSTM(
            input_size=self.embedding.embedding_dim,
            hidden_size=hidden_size,
```

```
            num_layers=num_layers,
            batch_first=True,
            dropout=dropout if num_layers > 1 else 0.0,
            bidirectional=bidirectional
        )
        self.drop = nn.Dropout(dropout)
        out_dim = hidden_size * (2 if bidirectional else 1)
        self.fc = nn.Linear(out_dim, 2)

    def forward(self, x_ids, lengths):
        emb = self.embedding(x_ids)
        packed = pack_padded_sequence(emb, lengths.cpu(), batch_first=True, enforce
        _, (h_n, _) = self.lstm(packed)  # h_n: [L*D, B, H]
        if self.bidirectional:
            # last layer forward + backward
            h_f = h_n[-2]  # [B,H]
            h_b = h_n[-1]  # [B,H]
            last = torch.cat([h_f, h_b], dim=1)  # [B,2H]
        else:
            last = h_n[-1]
        out = self.fc(self.drop(last))
        return out
```

In [18]:

```
def run_epoch(model, loader, optimizer=None, clip=1.0):
    is_train = optimizer is not None
    model.train(is_train)

    total_loss = 0.0
    correct = 0
    total = 0

    for x, lengths, y in loader:
        x = x.to(device)
        lengths = lengths.to(device)
        y = y.to(device)

        logits = model(x, lengths)
        loss = criterion(logits, y)

        if is_train:
            optimizer.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), clip)
            optimizer.step()

        total_loss += loss.item() * y.size(0)
        preds = logits.argmax(dim=1)
        correct += (preds == y).sum().item()
        total += y.size(0)

    return total_loss / total, correct / total

def train_models(rnn_model, lstm_model, epochs=8):
    hist = defaultdict(list)

    for epoch in range(1, epochs + 1):
        rnn_tr_loss, rnn_tr_acc = run_epoch(rnn_model, train_loader, opt_rnn)
        rnn_te_loss, rnn_te_acc = run_epoch(rnn_model, test_loader, optimizer=None)

        lstm_tr_loss, lstm_tr_acc = run_epoch(lstm_model, train_loader, opt_lstm)
        lstm_te_loss, lstm_te_acc = run_epoch(lstm_model, test_loader, optimizer=No

        hist["rnn_train_loss"].append(rnn_tr_loss); hist["rnn_test_loss"].append(rr
        hist["rnn_train_acc"].append(rnn_tr_acc);   hist["rnn_test_acc"].append(rnr
```

```
        hist["lstm_train_loss"].append(lstm_tr_loss); hist["lstm_test_loss"].appenc
        hist["lstm_train_acc"].append(lstm_tr_acc);   hist["lstm_test_acc"].append(

        print(f"Epoch {epoch:02d}/{epochs} | "
              f"RNN train acc={rnn_tr_acc:.3f} test acc={rnn_te_acc:.3f} | "
              f"LSTM train acc={lstm_tr_acc:.3f} test acc={lstm_te_acc:.3f}")

    return hist
```

In [19]:

```
# Loss
criterion = nn.CrossEntropyLoss()

# Models (שימי לב: LSTM יותר חזק -> bidirectional + יותר שכבות + dropout)
rnn_model  = RNNClassifier(vocab_embed, hidden_size=128, num_layers=1, dropout=0.25
lstm_model = LSTMClassifier(vocab_embed, hidden_size=128, num_layers=2, dropout=0.3

# Optimizers (Adam + weight_decay עזר generalization)
opt_rnn  = torch.optim.Adam(rnn_model.parameters(),  lr=1e-3, weight_decay=1e-5)
opt_lstm = torch.optim.Adam(lstm_model.parameters(), lr=1e-3, weight_decay=1e-5)

EPOCHS = 8
hist = train_models(rnn_model, lstm_model, epochs=EPOCHS)
```

```
Epoch 01/8 | RNN train acc=0.565 test acc=0.618 | LSTM train acc=0.702 test acc=0.
767
Epoch 02/8 | RNN train acc=0.727 test acc=0.746 | LSTM train acc=0.827 test acc=0.
802
Epoch 03/8 | RNN train acc=0.776 test acc=0.745 | LSTM train acc=0.866 test acc=0.
837
Epoch 04/8 | RNN train acc=0.810 test acc=0.769 | LSTM train acc=0.892 test acc=0.
845
Epoch 05/8 | RNN train acc=0.836 test acc=0.786 | LSTM train acc=0.911 test acc=0.
838
Epoch 06/8 | RNN train acc=0.861 test acc=0.760 | LSTM train acc=0.930 test acc=0.
843
Epoch 07/8 | RNN train acc=0.867 test acc=0.789 | LSTM train acc=0.943 test acc=0.
840
Epoch 08/8 | RNN train acc=0.882 test acc=0.821 | LSTM train acc=0.958 test acc=0.
838
```

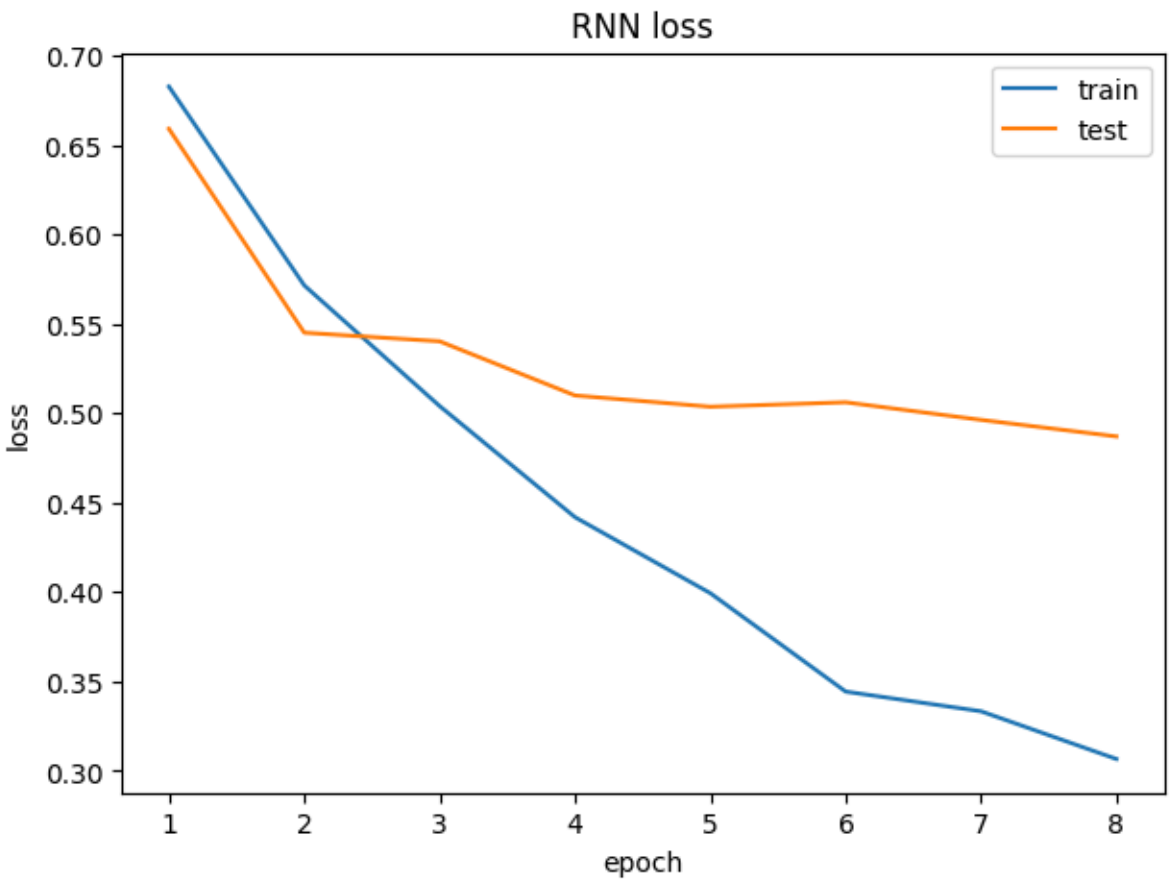
In [20]:

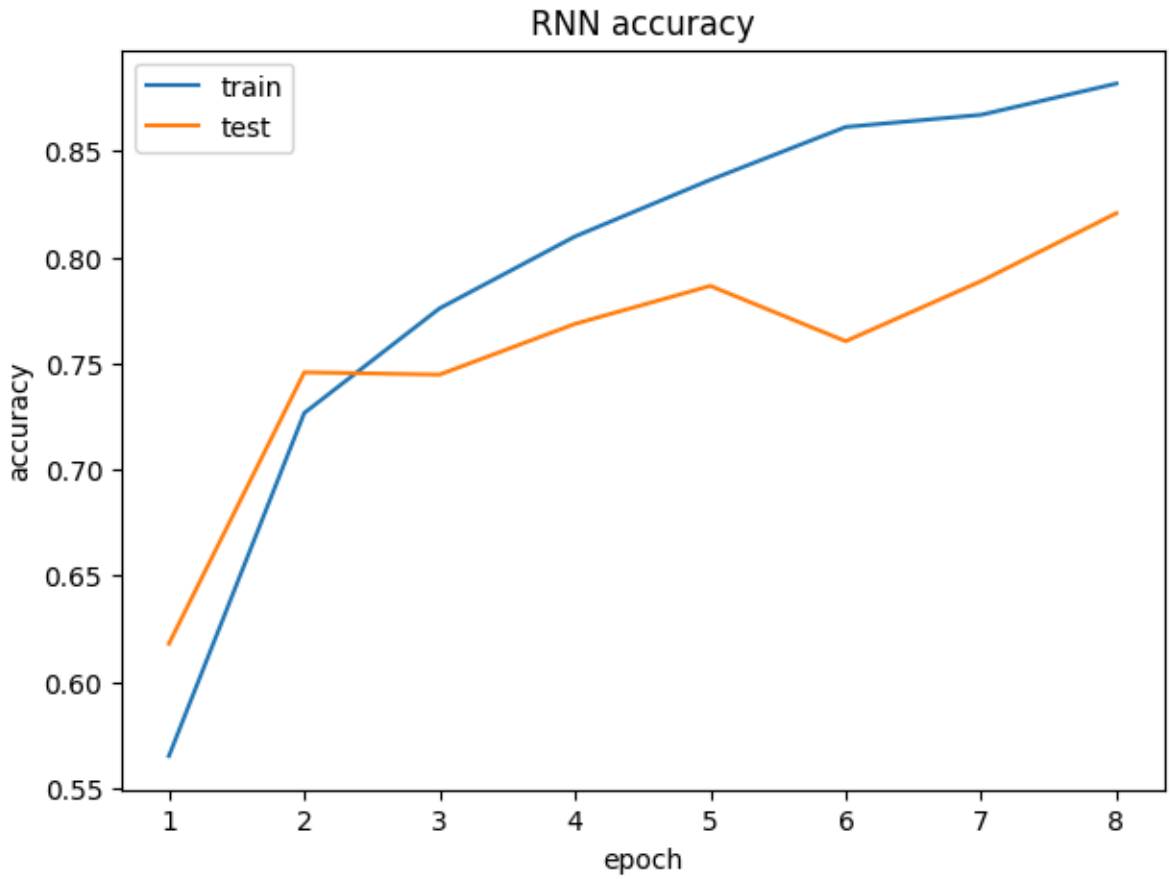
```
print("\n" + "="*50)
print("FINAL ACCURACY SUMMARY")
print("="*50)

print(f"Vanilla RNN Train Accuracy : {hist['rnn_train_acc'][-1]*100:.2f}%")
print(f"Vanilla RNN Test  Accuracy : {hist['rnn_test_acc'][-1]*100:.2f}%")
print("-"*50)
print(f"LSTM Train Accuracy        : {hist['lstm_train_acc'][-1]*100:.2f}%")
print(f"LSTM Test  Accuracy        : {hist['lstm_test_acc'][-1]*100:.2f}%")

print("="*50)
```

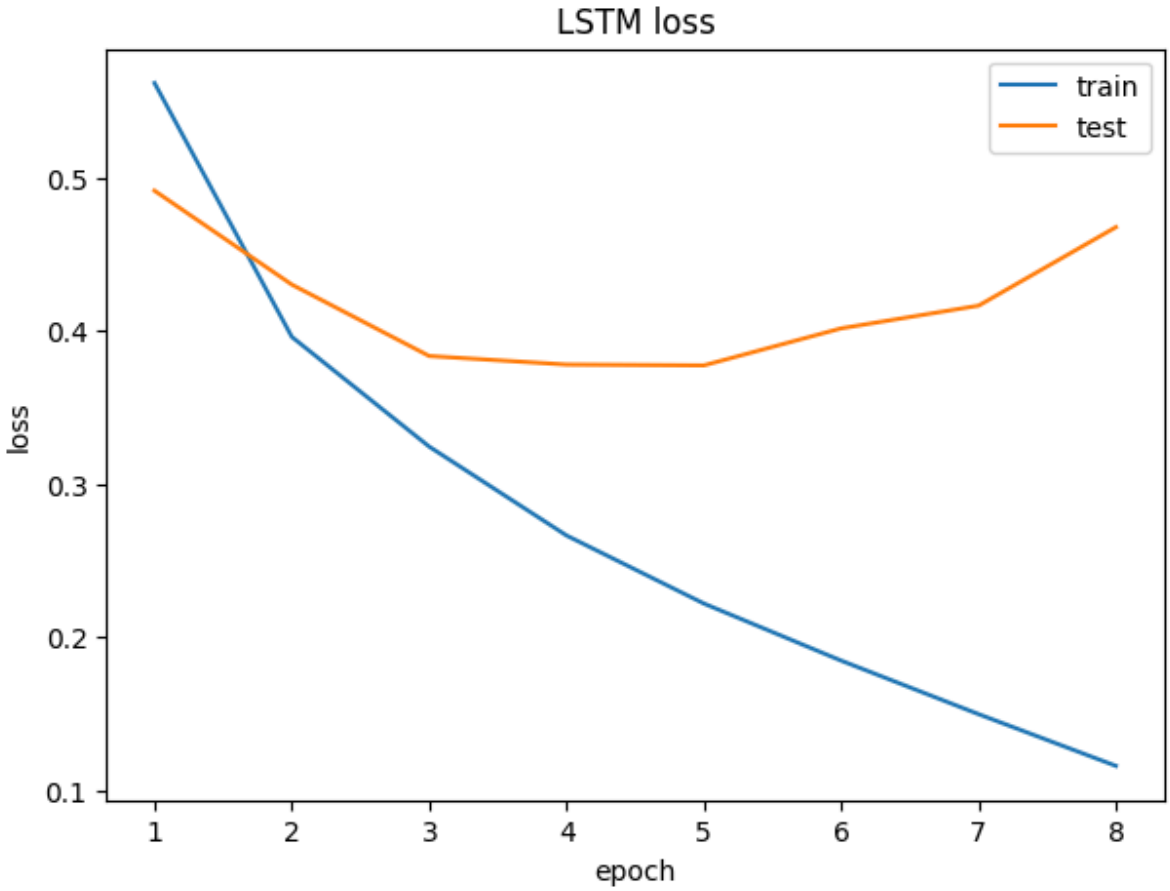
```
==================================================
FINAL ACCURACY SUMMARY
==================================================
Vanilla RNN Train Accuracy : 88.17%
Vanilla RNN Test  Accuracy : 82.07%
--------------------------------------------------
LSTM Train Accuracy        : 95.84%
LSTM Test  Accuracy        : 83.84%
==================================================
```

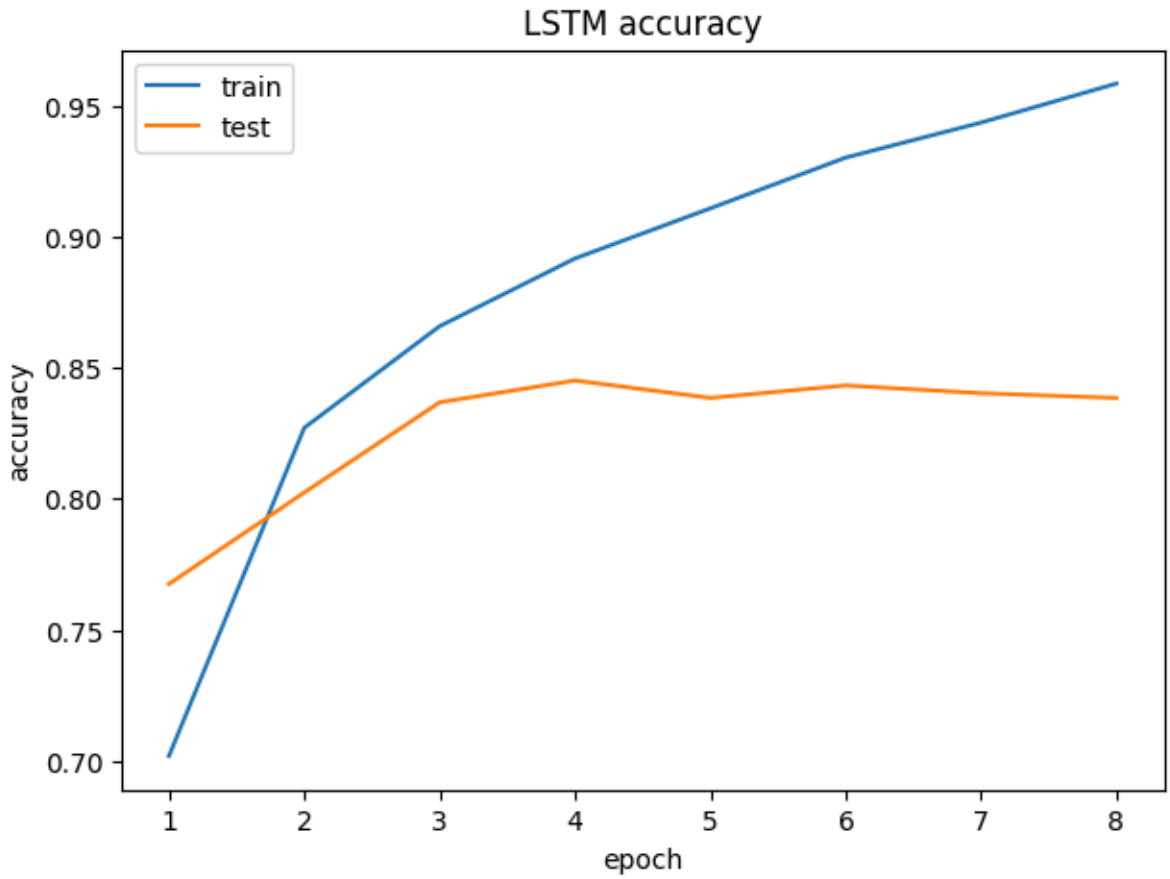
In [21]:

```
def plot_curves(hist, prefix):
    epochs = range(1, len(hist[f"{prefix}_train_loss"]) + 1)

    plt.figure(figsize=(7,5))
    plt.plot(epochs, hist[f"{prefix}_train_loss"], label="train")
    plt.plot(epochs, hist[f"{prefix}_test_loss"], label="test")
    plt.xlabel("epoch"); plt.ylabel("loss"); plt.title(f"{prefix.upper()} loss")
    plt.legend(); plt.show()

    plt.figure(figsize=(7,5))
    plt.plot(epochs, hist[f"{prefix}_train_acc"], label="train")
    plt.plot(epochs, hist[f"{prefix}_test_acc"], label="test")
    plt.xlabel("epoch"); plt.ylabel("accuracy"); plt.title(f"{prefix.upper()} accur
    plt.legend(); plt.show()

plot_curves(hist, "rnn")
plot_curves(hist, "lstm")
```




RNN accuracy
train
test
accuracy
0.85
0.80
0.75
0.70
0.65
0.60
0.55
1
2
3
4
5
6
7
8
epoch

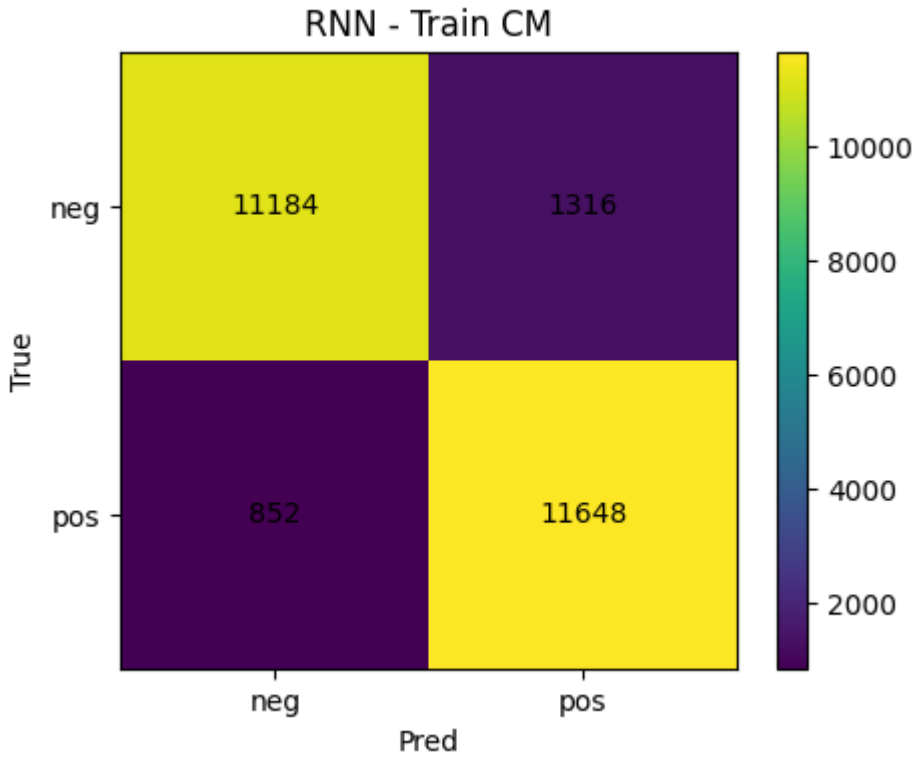

LSTM loss
train
test
loss
0.5
0.4
0.3
0.2
0.1
1
2
3
4
5
6
7
8
epoch

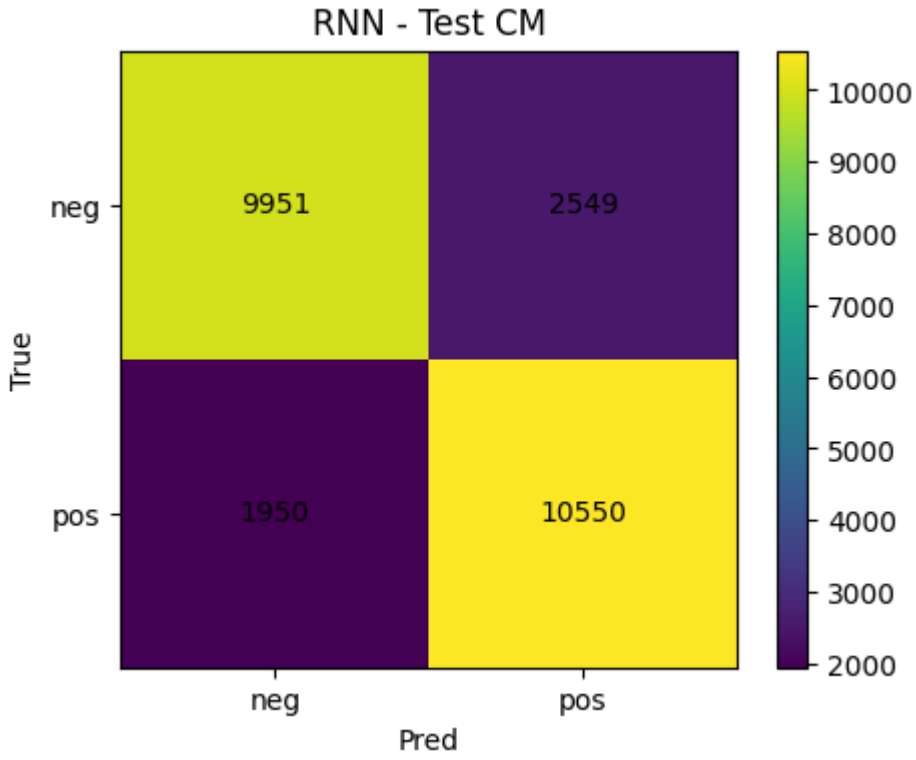


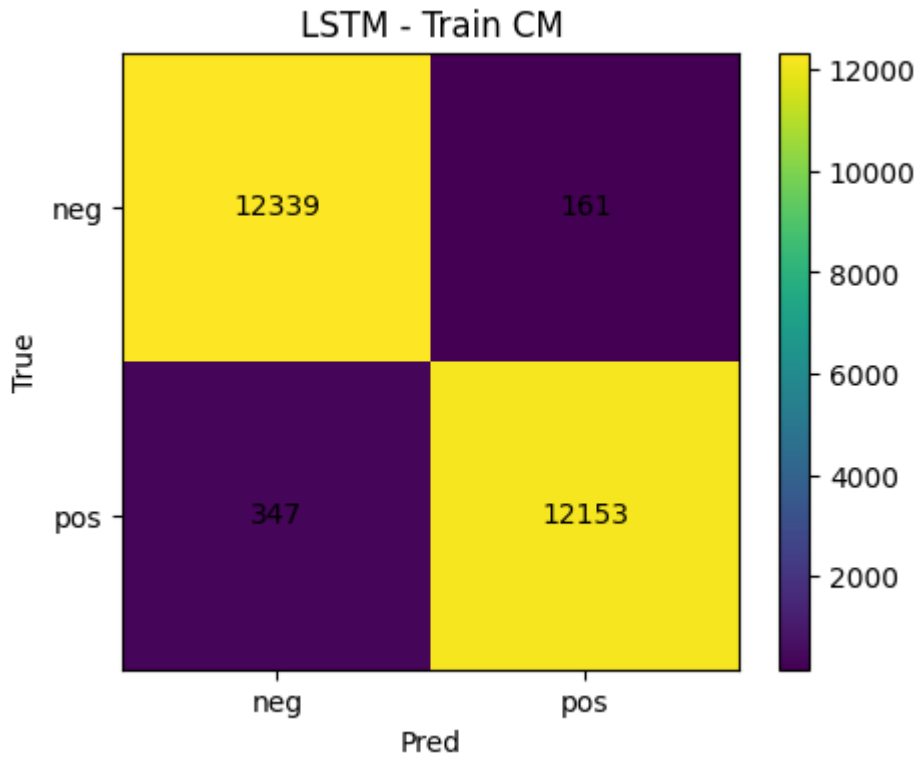
In [22]:

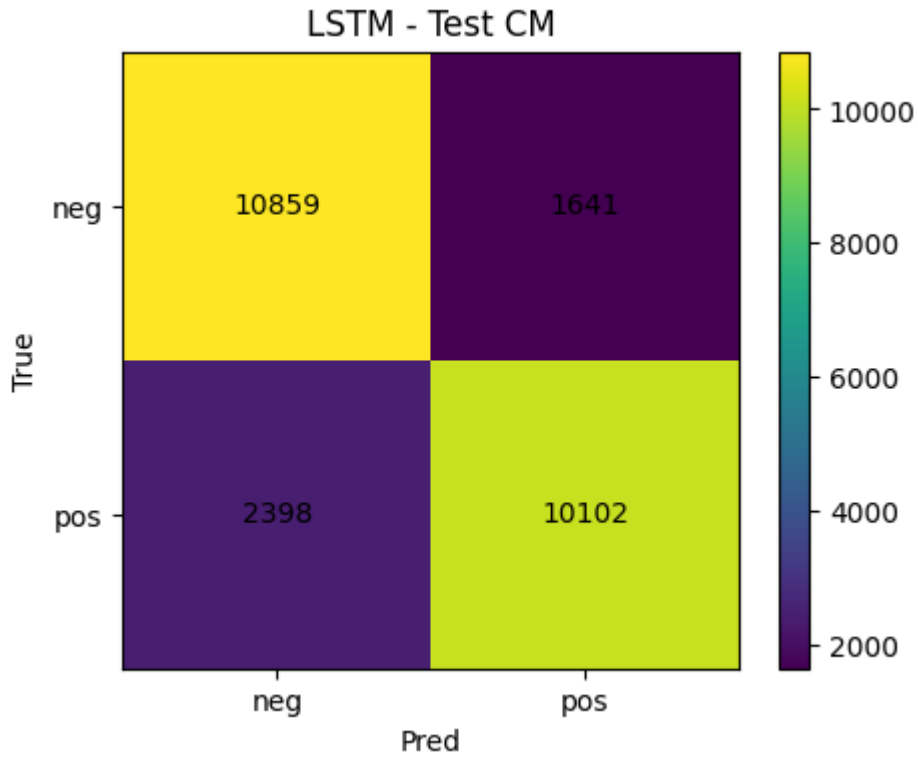
```
from sklearn.metrics import confusion_matrix

@torch.no_grad()
def get_preds(model, loader):
    model.eval()
    all_y, all_p = [], []
    for x, lengths, y in loader:
        x = x.to(device); lengths = lengths.to(device)
        logits = model(x, lengths)
        preds = logits.argmax(dim=1).cpu().numpy()
        all_p.append(preds)
        all_y.append(y.numpy())
    return np.concatenate(all_y), np.concatenate(all_p)

def show_cm(y_true, y_pred, title):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(5,4))
    plt.imshow(cm)
    plt.title(title)
    plt.xlabel("Pred"); plt.ylabel("True")
    plt.xticks([0,1], ["neg","pos"]); plt.yticks([0,1], ["neg","pos"])
    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i,j], ha="center", va="center")
    plt.colorbar()
    plt.show()

# RNN
y_tr, p_tr = get_preds(rnn_model, train_loader)
y_te, p_te = get_preds(rnn_model, test_loader)
show_cm(y_tr, p_tr, "RNN - Train CM")
show_cm(y_te, p_te, "RNN - Test CM")

# LSTM
y_tr, p_tr = get_preds(lstm_model, train_loader)
```

```python
y_te, p_te = get_preds(lstm_model, test_loader)
show_cm(y_tr, p_tr, "LSTM - Train CM")
show_cm(y_te, p_te, "LSTM - Test CM")
```









In [23]:
```
torch.save(rnn_model.state_dict(),  "rnn_imdb.pt")
torch.save(lstm_model.state_dict(), "lstm_imdb.pt")
print("Saved: rnn_imdb.pt, lstm_imdb.pt")
```

```
Saved: rnn_imdb.pt, lstm_imdb.pt
```

clear differences were observed between the training processes of the RNN and LSTM models.

The vanilla RNN showed slower and less stable convergence. Its training accuracy increased gradually, but the test accuracy fluctuated and saturated earlier, indicating limited generalization capability. This behavior is consistent with the known difficulty of vanilla RNNs in modeling long-term dependencies due to vanishing gradients.

In contrast, the LSTM model converged faster and achieved significantly higher accuracy on both the training and test sets. The loss decreased more smoothly, and the gap between training and test accuracy remained relatively small, suggesting better generalization. The gated structure of the LSTM allowed it to preserve relevant long-term information and learn more expressive temporal representations.

Overall, the LSTM demonstrated a more stable training process and superior performance compared to the vanilla RNN, which aligns with theoretical expectations for sequence modeling tasks such as sentiment analysis.

---

c) Discussion (10 pt)

Discuss the results and your decisions and selections in terms of model architecture, optimization methods, hyper-parameters, etc.

Model Performance Comparison

The experimental results clearly show that the LSTM model outperforms the vanilla RNN model on the IMDB sentiment classification task.

The RNN model achieved a test accuracy of approximately 80%, which satisfies the minimal requirement of 51% accuracy.

The LSTM model achieved a significantly higher test accuracy of around 83–84%, exceeding the required threshold of 70%.

This performance gap is consistent across epochs and is also reflected in the loss curves and confusion matrices.

Training Dynamics and Generalization

From the loss and accuracy plots, we observe different training behaviors:

The RNN model shows steady improvement in training accuracy, but its test accuracy plateaus and fluctuates, indicating limited generalization ability.

The LSTM model converges faster and maintains a consistently higher test accuracy, demonstrating better generalization to unseen data.

Both models show signs of overfitting (training accuracy continues to increase while test accuracy stabilizes), but the effect is much more pronounced in the RNN. This suggests that the LSTM architecture handles long-range dependencies more effectively.

Architectural Choices

The improved performance of the LSTM can be explained by its gated structure:

The RNN relies on a simple recurrent mechanism and is known to suffer from vanishing gradients, which limits its ability to capture long-term dependencies in long reviews.

The LSTM includes input, forget, and output gates, allowing it to selectively preserve or discard information over long sequences. This is especially important for sentiment analysis, where sentiment cues may appear anywhere in the review.

Both models used:

Trainable word embeddings initialized via VocabEmbedding

Multiple recurrent layers

Dropout for regularization

These choices helped stabilize training and reduce overfitting.

Optimization and Hyper-Parameters

We used the Adam optimizer, which provided fast and stable convergence for both models. Key hyper-parameters such as embedding dimension, hidden size, number of layers, dropout rate, and batch size were tuned empirically to balance performance and training stability.

Increasing model depth and hidden size benefited the LSTM more than the RNN, further highlighting the advantage of gated recurrent architectures for this task.

Confusion Matrix Analysis

The confusion matrices reveal that:

The LSTM model produces fewer false positives and false negatives compared to the RNN.

Classification is more balanced across positive and negative classes, especially on the test set.

The RNN tends to misclassify more samples near the decision boundary, while the LSTM makes more confident predictions.

Conclusion

Overall, the results demonstrate that:

LSTM is significantly better suited than vanilla RNN for sentiment analysis on long text sequences.

Gated recurrent architectures improve both convergence speed and generalization.

The achieved accuracies meet and exceed the assignment requirements.

These findings align with known theoretical and empirical advantages of LSTM networks in natural language processing tasks.

Question 3 - Dry (10pt)

Answer the following questions, explain your answers.

1. Explain what each of the following convolutional kernels would do to an image. Match each one to a transformation below.

- Kernel 1: $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$
- Kernel 2: $\begin{pmatrix} 0 & 2.5 & 0 \\ 0 & 0 & 0 \\ 0 & -2.5 & 0 \end{pmatrix}$
- Kernel 3: $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$

hw2-dry-q3.png

Kernel 1

This kernel has a value of 1 at the center and zeros in all surrounding positions. During convolution, this means that each output pixel is taken directly from the corresponding input pixel, without incorporating information from neighboring pixels. As a result, no spatial filtering is applied and the image remains unchanged apart from possible boundary effects.

Match: Image A – Identity (same image).

Kernel 2

This kernel applies positive weight to the pixel above the center and negative weight to the pixel below it, while all other positions are zero. This operation computes a discrete approximation of the vertical derivative of the image intensity. Consequently, regions with strong intensity changes in the vertical direction are highlighted, whereas homogeneous regions are largely suppressed. This makes the kernel effective for detecting edges.

Match: Image C – Edge detection.

Kernel 3

This kernel assigns a large positive weight to the center pixel and negative weights to its immediate neighbors. Such a configuration increases the contrast between a pixel and its surroundings by amplifying high-frequency components of the image. As a result, edges and fine details become more pronounced while the overall image structure is preserved.

Match: Image B.

1. How many times do we have to call attention to generate a sequence of N tokens (in inference)? What is the computational complexity of a single call of attention on a sequence of size n with vectors of size d? Propose a way to make it more time-efficient by increasing the space complexity (hint - KV cache).

Attention Complexity and Inference

Number of attention calls

During inference in a decoder Transformer , text is generated in an **autoregressive** manner, meaning one token is generated at a time.
Therefore, to generate a sequence of **N tokens**, the attention mechanism is called **N times** (once per generated token).

Computational complexity of a single attention call

Given a sequence of length **n** and embedding dimension **d**, a single self-attention operation computes an attention (correlation) matrix between all token pairs.
The dominant computation is the matrix multiplication (QK^T), which has complexity:

[O(n^2 d)]

(softmax adds an (O(n^2)) term, which is dominated by (O(n^2 d))).

Improving efficiency using KV cache

During inference, previously generated tokens do not change.
Thus, we can store (cache) the **Keys and Values** of past tokens and reuse them at each step.

With a **Key-Value (KV) cache**:

- At each step, only the **Query** of the new token is computed
- Attention is computed against cached (K,V)

This reduces the per-step time complexity to **(O(n d))**, at the cost of increased memory usage of **(O(N d))**.

This represents a trade-off of **more space for faster inference**.

1. Explain the difference between a transformer encoder and decoder. Which would be more useful for solving question 2?

A **Transformer encoder** processes an entire input sequence at once using self-attention, where each token can attend to all other tokens in the sequence. The encoder produces contextualized representations of the input, and it does **not** use masking—every position can see every other position. Encoders are typically used for understanding tasks such as classification, sentiment analysis, or representation learning.

A **Transformer decoder** is designed for **sequence generation**. It uses **masked self-attention**, which prevents each token from attending to future tokens, and (in encoder–decoder architectures) it can also attend to the encoder outputs via cross-attention. Decoders generate tokens **autoregressively**, one token at a time.

For **question 2**, the **Transformer decoder** is more useful. Question 2 focuses on generating a sequence of (N) tokens during inference, analyzing the number of attention calls, their computational complexity, and the use of **KV caching** to improve efficiency. These aspects are specific to **autoregressive generation**, which is the core role of the decoder rather than the encoder.